

Ex 5:

```
void dfs- aux (Grafo *g, int atual, int *visitado) {
```

```
    visitado[atual] = 1
```

```
    printf("/d ", g->v[atual]);
```

```
    for (int i = 0; i < g->n; i++) {
```

```
        if (g->m[atual][i] != 0 && !visitado[i]) {
```

```
            dfs- aux(g, i, visitado);
```

```
        }
```

```
    }
```

```
}
```

```
void dfs- recursiva (Grafo *g, int origem) {
```

```
    int *visitado = calloc(g->n, sizeof(int));
```

```
    dfs- aux(g, origem, visitado);
```

```
    free(visitado);
```

```
}
```

Ex 6:

```
int existe - caminhos - bfs (Grafo *g, int origem, int destino) {
```

```
    int *descobertos = calloc(g->n, sizeof(int));
```

```
    FILE *f = fopen("...", "w");
```

```
    enfileirar(f, origem);
```

```
descoberto[origem] = 1;
```

```
while (!finalizada(f)) {  
    int atual;  
    desempilha(f, &atual);  
    if (atual == destino) {  
        free(descoberto);  
        destruirFile(f);  
        return 1;  
    }  
}
```

```
for (int i = 0; i < g->n; i++) {  
    if (g->m[atual][i] != g->arestaula && !descoberto[i]) {  
        empilha(f, i);  
        descoberto[i] = 1;  
    }  
}
```

```
free(descoberto);  
destruirFile(f);  
return 0;
```

Ex 7:

```
int dfs-convex (Grafo *g, int atual, int *descoberto) {  
    descoberto[atual] = 1;  
    int total = 1;  
    for (int i = 0; i < g->n; i++) {  
        if (g->m[atual][i] != g->arestaula && !descoberto[i]) {  
            total += dfs-convex(g, i, descoberto);  
        }  
    }  
}
```

```

    }
    return total;
}

```

```

int contar_alcancaes (Grafo * g, int origem) {
    int *descoberto = calloc (g->n, sizeof(int));
    int total = dfs-contrar (g, origem, descoberto);
    free (descoberto);
    return total;
}

```

x 8:

```

int * obter_adjacentes (Grafo * g, int vertice, int * quantidade) {
    *quantidade = 0;
    for (int i = 0; i < g->n; i++) {
        if (g->m[vertices][i] != g->arestanula) {
            (*quantidade)++;
        }
    }

    int *adj = malloc ((*quantidade) * sizeof(int));
    int pos = 0;
    for (int i = 0; i < g->n; i++) {
        if (g->m[vertices][i] != g->arestanula) {
            adj[pos++] = g->v[i];
        }
    }

    return adj;
}

```

Ex 9:

```
int eh-isolado (Grafo *g, int vertice) {  
    for (int i=0; i < g->n; i++) {  
        if (g->m[vertice][i] != g->arestasrule) {  
            return 0;  
        }  
    }  
    return 1;  
}
```

Ex 10:

```
int contare-arestas (Grafo *g) {  
    int total = 0;  
    for (int i=0; i < g->n; i++) {  
        for (int j=i+1; j < g->n; j++) {  
            if (g->m[i][j] != g->arestasrule) {  
                total++;  
            }  
        }  
    }  
    return total;  
}
```

Ex 11:

```
int grau-entrada (Grafo *g, int vertice) {  
    int grau = 0;  
    for (int i=0; i < g->n; i++) {  
        if (g->m[i][vertice] != g->arestasrule) {  
            grau++;  
        }  
    }  
    return grau;  
}
```

Ex 12:

```
int grau = void (Grafo *g, int vertice) {  
    int grau = 0;  
    for (int j = 0; j < g->n; j++) {  
        if (g->m[vertice][j] != g->arestas[0])  
            grau++;  
    }  
    return grau;  
}
```

Ex 13:

```
void marcar-componentes (Grafo *g, int actual, int *visitados) {  
    visitados[actual] = 1;  
    for (int i = 0; i < g->n; i++) {  
        if (g->m[actual][i] != g->arestas[0] && !visitados[i]) {  
            marcar-componentes(g, i, visitados);  
        }  
    }  
}
```

```
int contar-componentes (Grafo *g) {  
    int *visitados = calloc(g->n, sizeof(int));  
    int componentes = 0;  
    for (int i = 0; i < g->n; i++) {  
        if (!visitados[i]) {  
            componentes++;  
            marcar-componentes(g, i, visitados);  
        }  
    }  
    free(visitados);  
}
```

return components;

3

//

Ex 5:

```
void dfs-aux(Grafo *g, int atual, int *visitados) {
```

```
    visitados[atual] = 1;
```

```
    printf("/d ", &g->v[atual]);
```

```
    for (int i = 0; i < g->n; i++) {
```

```
        if (g->m[atual][i] != g->arestas_nulas && !visitados[i]) {
```

```
            dfs-aux(g, i, visitados);
```

```
        }
```

```
    }
```

```
}
```

```
void dfs-recursiva(Grafo *g, int origem) {
```

```
    int *visitados = calloc(g->n, sizeof(int));
```

```
    dfs-aux(g, origem, visitados);
```

```
    free(visitados);
```

```
}
```

Ex 6:

```
int existe_caminho_dfs(Grafo *g, int origem, int destino) {
```

```
    int *descobertos = calloc(g->n, sizeof(int));
```

```
    File *f = criarFile();
```

```
    empilhar(f, origem);
```

```
    descobertos[origem] = 1;
```

```
    while (!file_vazio(f)) {
```

```
        int atual;
```

```
        desempilhar(f, &atual);
```

```
        if (atual == destino) {
```

```
            free(descobertos);
```

```

    destruirFile(f);
    return 1;
}

```

```

for (int i=0; i<g->n; i++) {
    if (g->m[atual][i] != g->arestas && !descoberto[i]) {
        enfileirar(f, i);
        descoberto[i] = 1;
    }
}
}

```

```

}
free(descoberto);
destruirFile(f);
return 0;
}

```

Ex 7:

```

int dfs_contar(Grafo *g, int atual, int *descoberto) {
    descoberto[atual] = 1;
    int total = 1;
    for (int i=0; i<g->n; i++) {
        if (g->m[atual][i] != g->arestas && !descoberto[i]) {
            total += dfs_contar(g, i, descoberto);
        }
    }
}

```

```

}
return total;
}

```

```

int contar_alcance(Grafo *g, int origem) {
    int *descoberto = calloc(g->n, sizeof(int));
    int total = dfs_contar(g, origem, descoberto);
    free(descoberto);
    return total;
}

```

ex 13:

```
void marcar-componentes (Grafo *g, int atual, int *visitados){  
    visitados[atual] = 1;  
    for (int i = 0; i < g->n; i++){  
        if (g->m[atual][i] != g->arestas & & !visitados[i]){  
            marcar-componentes (g, i, visitados);  
        }  
    }  
}
```

```
int contar-componentes (Grafo *g){  
    int *visitados = calloc(g->n, sizeof(int));  
    int componentes = 0;  
    for (int i = 0; i < g->n; i++){  
        if (!visitados[i]){  
            componentes++;  
            marcar-componentes (g, i, visitados);  
        }  
    }  
    free (visitados);  
    return componentes;  
}
```